JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels.spi

Class AbstractSelector

java.lang.Object
    java.nio.channels.Selector
        java.nio.channels.spi.AbstractSelector

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

```
public abstract class AbstractSelector
extends Selector
```

Base implementation class for selectors.

This class encapsulates the low-level machinery required to implement the interruption of selection operations. A concrete selector class must invoke the `begin` and `end` methods before and after, respectively, invoking an I/O operation that might block indefinitely. In order to ensure that the `end` method is always invoked, these methods should be used within a `try` ... `finally` block:

```
try {
    begin();
    // Perform blocking I/O operation here
    ...
} finally {
    end();
}
```

This class also defines methods for maintaining a selector's cancelled-key set and for removing a key from its channel's key set, and declares the abstract `register` method that is invoked by a selectable channel's `register` method in order to perform the actual work of registering a channel.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`AbstractSelector(SelectorProvider provider)`	Initializes a new instance of this class.

Method Summary

Concrete Methods

Modifier and Type	Method	Descriptio
protected void	`begin()`	Marks the beginning of an I/O operation that migh block indefinitel
protected `Set<SelectionKey>`	`cancelledKeys()`	Retrieves this selector's cancelled- key set.
void	`close()`	Closes thi selector.
protected void	`deregister(AbstractSelectionKey key)`	Removes the given key from its channel's key set.
protected void	`end()`	Marks the end of an O

		operation that migh block indefinitel
protected abstract void	**implCloseSelector**()	Closes thi selector.
boolean	**isOpen**()	Tells whether o not this selector is open.
SelectorProvider	**provider**()	Returns the provider that created this channel.
protected abstract **SelectionKey**	**register**(**AbstractSelectableChannel** ch, int ops, **Object** att)	Registers the given channel with this selector.

Methods inherited from class java.nio.channels.Selector

keys, open, select, select, selectedKeys, selectNow, wakeup

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

AbstractSelector

```
protected AbstractSelector(SelectorProvider provider)
```

Initializes a new instance of this class.

Parameters:

`provider` - The provider that created this selector

Method Detail

close

```
public final void close()
                throws IOException
```

Closes this selector.

If the selector has already been closed then this method returns immediately. Otherwise it marks the selector as closed and then invokes the `implCloseSelector` method in order to complete the close operation.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in class `Selector`

Throws:

`IOException` - If an I/O error occurs

implCloseSelector

```
protected abstract void implCloseSelector()
                                    throws IOException
```

Closes this selector.

This method is invoked by the `close` method in order to perform the actual work of closing the selector. This method is only invoked if the selector has not yet been closed, and it is never invoked more than once.

An implementation of this method must arrange for any other thread that is blocked in a selection operation upon this selector to return immediately as if by invoking the wakeup method.

Throws:

IOException - If an I/O error occurs while closing the selector

isOpen

```
public final boolean isOpen()
```

Description copied from class: Selector

Tells whether or not this selector is open.

Specified by:

isOpen in class Selector

Returns:

true if, and only if, this selector is open

provider

```
public final SelectorProvider provider()
```

Returns the provider that created this channel.

Specified by:

provider in class Selector

Returns:

The provider that created this channel

cancelledKeys

```
protected final Set<SelectionKey> cancelledKeys()
```

Retrieves this selector's cancelled-key set.

This set should only be used while synchronized upon it.

Returns:

The cancelled-key set

register

```
protected abstract SelectionKey register(AbstractSelectableChannel ch,
                                         int ops,
                                         Object att)
```

Registers the given channel with this selector.

This method is invoked by a channel's `register` method in order to perform the actual work of registering the channel with this selector.

Parameters:

`ch - The channel to be registered`

`ops - The initial interest set, which must be valid`

`att - The initial attachment for the resulting key`

Returns:

`A new key representing the registration of the given channel with this selector`

deregister

```
protected final void deregister(AbstractSelectionKey key)
```

Removes the given key from its channel's key set.

This method must be invoked by the selector for each channel that it deregisters.

Parameters:

`key - The selection key to be removed`

begin

```
protected final void begin()
```

Marks the beginning of an I/O operation that might block indefinitely.

This method should be invoked in tandem with the `end` method, using a `try ... finally` block as shown above, in order to implement interruption for this selector.

Invoking this method arranges for the selector's `wakeup` method to be invoked if a thread's `interrupt` method is invoked while the thread is blocked in an I/O operation upon the selector.

end

```
protected final void end()
```

Marks the end of an I/O operation that might block indefinitely.

This method should be invoked in tandem with the `begin` method, using a `try` ... `finally` block as shown above, in order to implement interruption for this selector.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.